UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

Diseño de una Solución Concurrente: Estacionamiento Inteligente

Análisis de Memoria Compartida, Gestión del Recurso y Sección Crítica

Integrantes: Bryan Grogan • Xavier Cárdenas

Facultad de Ingeniería • 2026

Contextualización del Escenario

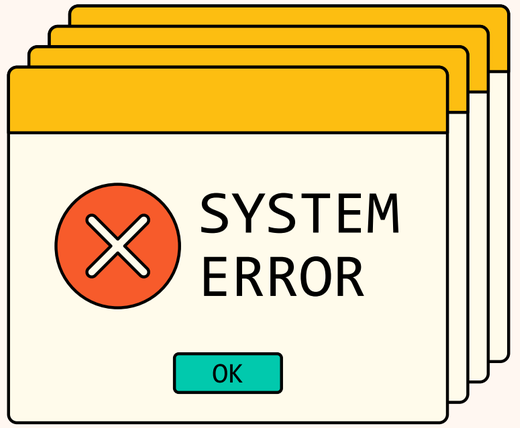
01
Escenario Principal
Sistema automatizado de control de acceso y permanencia en un parqueo con capacidad limitada de vehículos.

02
Ejecución Concurrente
Cada vehículo en tránsito se representa mediante un hilo independiente (threading.Thread) ejecutado en paralelo.

03
Desafío del Sistema
Coordinar múltiples ingresos y salidas simultáneas evitando inconsistencias en los datos y sobrecupo.







El Recurso Compartido y Necesidad de Coordinación

¿Qué representa el Recurso?

- Representa la disponibilidad real y el estado físico de los espacios de parqueo.
- Almacena qué espacios están libres/ocupados y la capacidad disponible restante.
- Es accedido simultáneamente por múltiples hilos de vehículos para consultar o modificar.

¿Por qué requiere Coordinación?

- Conflicto de Acceso: Dos hilos intentan reservar la misma plaza en el mismo milisegundo.
- Condición de Carrera (Race Condition): Lectura de disponibilidad desactualizada.
- Sobrecupo: Permitir el ingreso de más vehículos de la capacidad máxima configurada.
- Inconsistencia: Marcar un puesto libre mientras otro vehículo lo está ocupando.

Alternativas de Representación del Recurso en Python

1. Tipo Básico: Entero (int)

• Estructura: espacios_disponibles = capacidad
• Mecanismo: Modificación directa mediante -= 1 al ingresar y += 1 al salir.
• Ventajas: Implementación muy simple, mínima memoria y rápida ejecución.
• Limitación: Solo cuenta cantidad de cupos. No identifica qué vehículo ocupa cada plaza ni su tiempo.

2. Tipo Complejo: Diccionario (dict)

• Estructura: {puesto_id: {'ocupado': bool, 'vehiculo': str, 'hora': float}}
• Mecanismo: Asignación y liberación de plazas específicas dentro del diccionario.
• Ventajas: Trazabilidad completa, identificación exacta de puestos y tiempos.
• Limitación: Requiere mayor sincronización e iteración sobre la estructura.

Arquitectura de Clases y Relación con Hilos

Clase Estacionamiento

Administrador del Recurso

• Atributos: capacidad_maxima, puestos (dict), lock, semaforo
• Métodos: ingresar(vehiculo_id), salir(puesto_id)
• Responsabilidad: Proteger la sección crítica y garantizar exclusión mutua.

Clase Vehiculo

Unidad de Ejecución (Hilo)

• Atributos: vehiculo_id, estacionamiento (instancia compartida)
• Métodos: run()
• Responsabilidad: Solicitar parqueo, simular permanencia y solicitar salida.

Clase Simulador

Orquestador del Sistema

• Atributos: lista de hilos Vehiculo, objeto Estacionamiento
• Métodos: iniciar_simulacion(), reportar_resultados()
• Responsabilidad: Instanciar el recurso compartido y lanzar la ejecución.

Código Python: Representación Tipo Básico (int)

```
import threading

class EstacionamientoBasico:
    def __init__(self, capacidad):
        self.capacidad_maxima = capacidad
        self.espacios_disponibles = capacidad  # Recurso compartido tipo básico (int)
        self.lock = threading.Lock()           # Mecanismo de sincronización

    def ingresar(self, vehiculo_id):
        with self.lock:  # Inicio de la Sección Crítica
            if self.espacios_disponibles > 0:
                self.espacios_disponibles -= 1
                print(f"[{vehiculo_id}] Ingreso exitoso. Libres: {self.espacios_disponibles}")
                return True
            print(f"[{vehiculo_id}] Estacionamiento lleno.")
            return False

    def salir(self, vehiculo_id):
        with self.lock:  # Inicio de la Sección Crítica
            self.espacios_disponibles += 1
            print(f"[{vehiculo_id}] Salida registrada. Libres: {self.espacios_disponibles}")
```

Código Python: Representación Tipo Complejo (dict)

```
import threading

class EstacionamientoComplejo:
    def __init__(self, capacidad):
        self.capacidad_maxima = capacidad
        # Recurso compartido tipo complejo (dict)
        self.puestos = {i: {"ocupado": False, "vehiculo": None} for i in range(1, capacidad + 1)}
        self.lock = threading.Lock()

    def ingresar(self, vehiculo_id):
        with self.lock:  # Sección Crítica: Búsqueda y reserva exclusiva
            for puesto_id, datos in self.puestos.items():
                if not datos["ocupado"]:
                    datos["ocupado"] = True
                    datos["vehiculo"] = vehiculo_id
                    return puesto_id  # Asignación exitosa
            return None

    def salir(self, puesto_id, vehiculo_id):
        with self.lock:  # Sección Crítica: Liberación exclusiva
            if self.puestos[puesto_id]["vehiculo"] == vehiculo_id:
                self.puestos[puesto_id]["ocupado"] = False
                self.puestos[puesto_id]["vehiculo"] = None
```

Código Python: Unidad de Ejecución (Clase Vehiculo)

```
import threading
import time
import random

class Vehiculo(threading.Thread):
    def __init__(self, vehiculo_id, estacionamiento_compartido):
        super().__init__()
        self.vehiculo_id = vehiculo_id
        # Todos los hilos reciben la MISMA instancia del recurso compartido
        self.estacionamiento = estacionamiento_compartido

    def run(self):
        # 1. Intentar ingresar a la sección protegida
        puesto = self.estacionamiento.ingresar(self.vehiculo_id)
        if puesto:
            print(f"--> [{self.vehiculo_id}] Estacionado en el puesto #{puesto}")
            time.sleep(random.uniform(1.0, 3.0))  # Simula tiempo de permanencia
            # 2. Liberar el espacio
            self.estacionamiento.salir(puesto, self.vehiculo_id)
            print(f"<-- [{self.vehiculo_id}] Liberó el puesto #{puesto} y se retiró.")
        else:
            print(f"xx  [{self.vehiculo_id}] Estacionamiento lleno. Retirándose.")
```

Identificación de la Sección Crítica

Instrucciones de la Sección Crítica

- Consulta de Disponibilidad: Verificar if self.espacios_disponibles > 0 o if not datos['ocupado'].
- Modificación del Estado: Decrementar contador (-= 1) o actualizar datos['ocupado'] = True.
- Liberación del Parqueo: Incrementar contador (+ space) o resetear valores del diccionario.

Inconsistencias por Ejecución Simultánea

- Doble Asignación: Dos hilos leen la misma plaza libre antes de actualizarla y se les asigna el mismo puesto.
- Violación del Límite de Capacidad: Múltiples hilos leen que queda 1 cupo y todos ingresan simultáneamente.
- Corrupción de la Estructura: Modificaciones simultáneas sobre el diccionario generan excepciones en tiempo de ejecución.

Mecanismo de Sincronización: Mutex y Semáforo

Exclusión Mutua con Mutex (Lock)

• Función: Garantiza que solo UN hilo ejecute el bloque de código de lectura/escritura a la vez.
• Adquisición: Se ejecuta lock.acquire() (o con el bloque with self.lock:) antes de consultar la disponibilidad.
• Liberación: Se ejecuta lock.release() automáticamente al salir del bloque condicional o método.

Control de Cupos con Semáforo

• Función: Semáforo contador inicializado con la capacidad máxima del parqueo.
• Bloqueo Eficiente: Si no hay puestos libres, el hilo solicitante se bloquea automáticamente sin consumir ciclos de CPU.
• Liberación: Al salir un vehículo, semaforo.release() despierta al siguiente hilo en cola de espera.

Selección Definitiva del Recurso y Justificación Final

Decisión: Se selecciona la Representación de Tipo Complejo (Diccionario)

- Asignación Específica de Plaza: En un sistema real de Estacionamiento Inteligente, se requiere indicar al conductor el número exacto de puesto asignado.
- Trazabilidad e Historial: Permite auditar qué vehículo ocupa cada plaza, registrar la hora exacta de ingreso y calcular la tarifa al salir.
- Escalabilidad del Sistema: Facilita la integración futura con paneles LED de señalización, sensores de presencia físicos o subsistemas de cobro por tiempo.
- Garantía de Consistencia: Protegido por el mecanismo Lock + Semáforo, el diccionario brinda un control robusto, elegante y seguro frente a accesos concurrentes.